CSC 251 Lecture 22 – Graphical User Interfaces (GUIs)

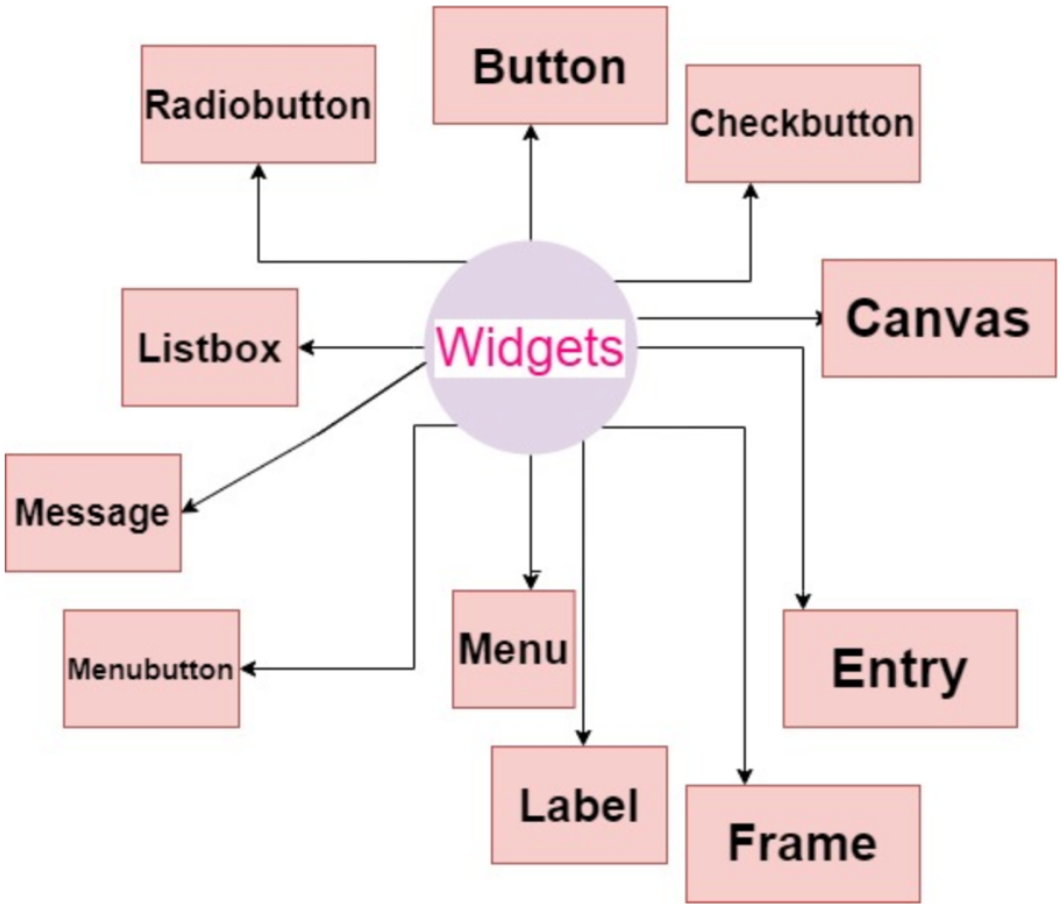
GUI Programming

Python packages:

- tkinter – simple tool GUIs, small portable applications
- PyQt and PySide – best for desktop applications, multimedia, scientific and engineering software
- Kivy – mobile app development
- PySimpleGUI, WxPython, Remi, and many more

tkinter

- Included in Python installation, a GUI library rather than a framework, cross-platform "Tk interface"
- Programs are event-driven based upon user input
- 19 graphical widgets through which the user may interact or view. Some of them:



https://docs.python.org/3/library/tkinter.html

Type: `python -m tkinter` at the command line to check that tkinter is properly installed.
If not, type: `sudo apt-get install python3-tk`

Best created as a class with an `__init__` method to build the GUI:

- Create a window object
- Create a main/root widget with a main window attribute
- Customize the widget
- Call the pack method to determine position and make the widget visible
- Call the `mainloop` function, an infinite loop that runs until the main window is closed

Example 1: Let's start with a window with a title:

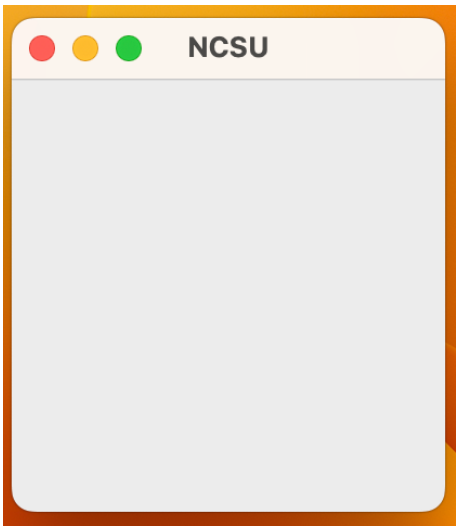
```
import tkinter as tk

class MyGUI:
    def __init__(self):
        # Create the main window
        self.main_window = tk.Tk()

        # Display a text on title bar
        self.main_window.title("NCSU")

        # Start the Tkinter event loop
        tk.mainloop()

if __name__ == "__main__":
    my_gui = MyGUI()
```



Close the window to terminate the program.

Label widgets

A label widget displays a single line of text in the window. Create an instance of the Label class, with the first argument indicating the root widget, the second with the text to be displayed.

Then call the Label widget's pack method to indicate its position in the root window (left, right, top, bottom). If no position is indicated, text labels are stacked in the center.

Example 2a:

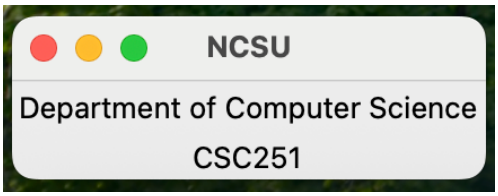
```
class MyGUI:
    def __init__(self):
        # Create the main window
        self.main_window = tk.Tk()

        #Create 2 Label widgets
        self.label1 = tk.Label(self.main_window,
                text='Department of Computer Science')
        self.label2 = tk.Label(self.main_window, text = 'CSC251')

        # Call pack methods
        self.label1.pack()
        self.label2.pack()

        # Display a text on title bar
        self.main_window.title("NCSU")

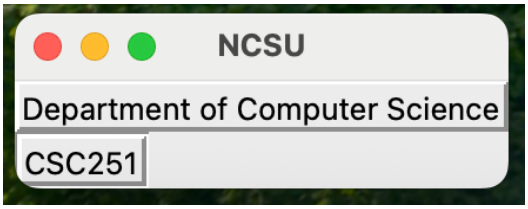
        # Start the Tkinter event loop
        tkinter.mainloop()
```



Example 2b: Add a border width (in pixels) and style (flat, raised, sunken, ridge, solid, groove):

```
self.label1 = tk.Label(self.main_window,  text= 'Department of Compute Science',
       borderwidth=2, relief='raised')
self.label2 = tk.Label(self.main_window, text = 'CSC251', borderwidth=3,
       relief = 'groove')

self.label1.pack(side = 'top')
self.label2.pack(side = 'left')
```

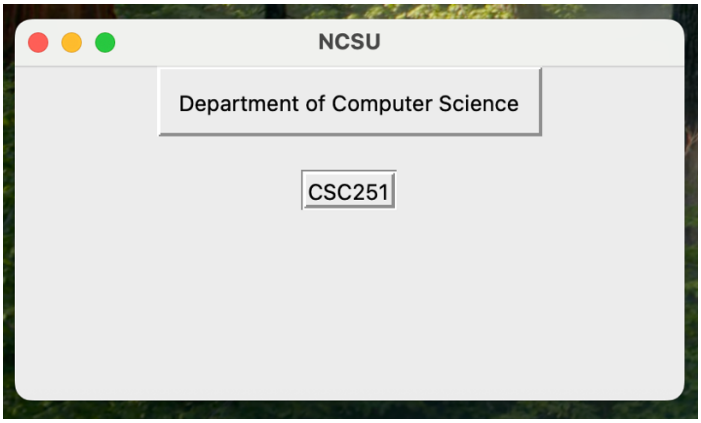


Example 3: (helloOOP1.py) Specify the size of the window in pixels:

```
self.main_window.geometry("400x200")
```

Add internal or external padding in terms of pixels

```
self.label1.pack(side = 'top', ipadx=10, ipady=10,)
self.label2.pack(side = 'top', padx=20, pady=20)
```



Button Widgets

Callback function (event handler) – a method that executes when the user clicks the button.
Import the tkinter messagebox module to use a function called `showinfo`:

```
import tkinter.messagebox
```

Example 4: (helloOOP2.py) Create the Button widget in the `__init__` method with a function call to a new method in the class and pack it to make it visible:

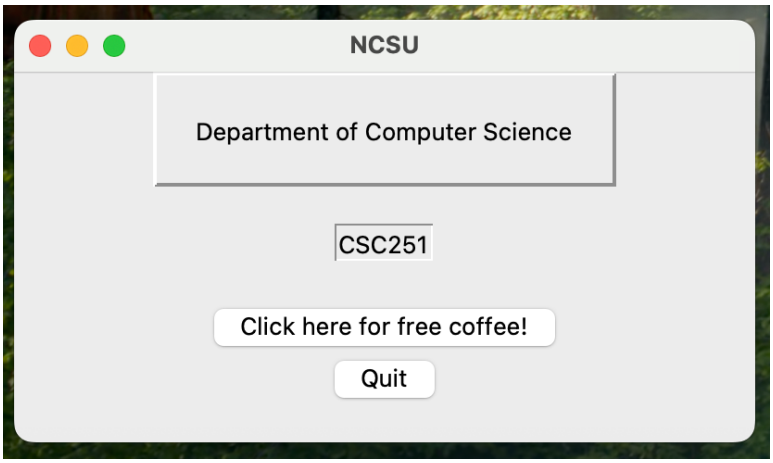
```
self.coffee_button = tk.Button(self.main_window,
       text = "Click here for free coffee!", command = self.wake_up)
self.coffee_button.pack()
```

The class method in which the showinfo method has title and message arguments

```
def wake_up(self):
    tk.messagebox.showinfo('Response', "Click Bait! Gotcha!")
```

Let's add a quit button to end the program and "destroy" the main window:

```
self.quit_button = tk.Button(self.main_window, text = 'Quit',
          command=self.main_window.destroy)
self.quit_button.pack()
```



Frames

Organize your widgets in frames, which is a container widget to hold other widgets.

Example 5: Create the frames and for each child widget, specify the frame it is contained within:

```
self.top_frame = tk.Frame(self.main_window)
self.bottom_frame = tk.Frame(self.main_window)
```

Pack the frames before the main loop:

```
self.top_frame.pack()
self.bottom_frame.pack()
```

Input with the Entry Widget

An entry widget provides a rectangular area for user input. Data typed into the widget can be retrieved. Often used along with a "submit" button.

Example 6: (helloOOPFrames.py) In the `__init__` method:

```
#Create an Entry widget for top frame

self.prompt_label = tk.Label(self.top_frame,
       text= 'Enter temperature in degrees F: ')
self.temp_entry = tk.Entry(self.top_frame, width = 10)

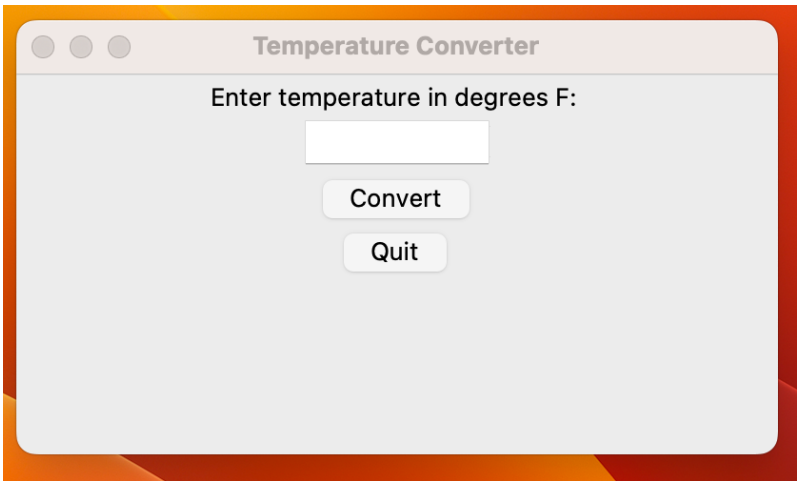
self.prompt_label.pack()
self.temp_entry.pack()

#Create a Button widget for bottom frame
self.calc_button = tk.Button(self.bottom_frame,  text= 'Convert', command =
       self.convert)
self.quit_button = tk.Button(self.bottom_frame, text = 'Quit', command=
       self.main_window.destroy)

self.quit_button.pack(side = 'bottom')
self.calc_button.pack(side = 'bottom')
```

Write a method for the class:

```
def convert(self):
    fahr = float(self.temp_entry.get())
    cel = (fahr - 32) * 5/9
    tkinter.messagebox.showinfo('Results', str(fahr)+' degrees fahrenheit = '
           + str(cel) + 'degrees celsius')
```



Improvements?

Labels as Output Fields

Example 7: (helloOOPFrames2.py) Let's get rid of the dialog box!
Add a middle frame:

```
self.mid_frame = tk.Frame(self.main_window)
```

Add a descriptive label:

```
self.descrip_label = tk.Label(self.mid_frame, text='Converted to degrees C: ')
```

Create a StringVar object to hold the output (will be set in the convert function) and a label to go with it:

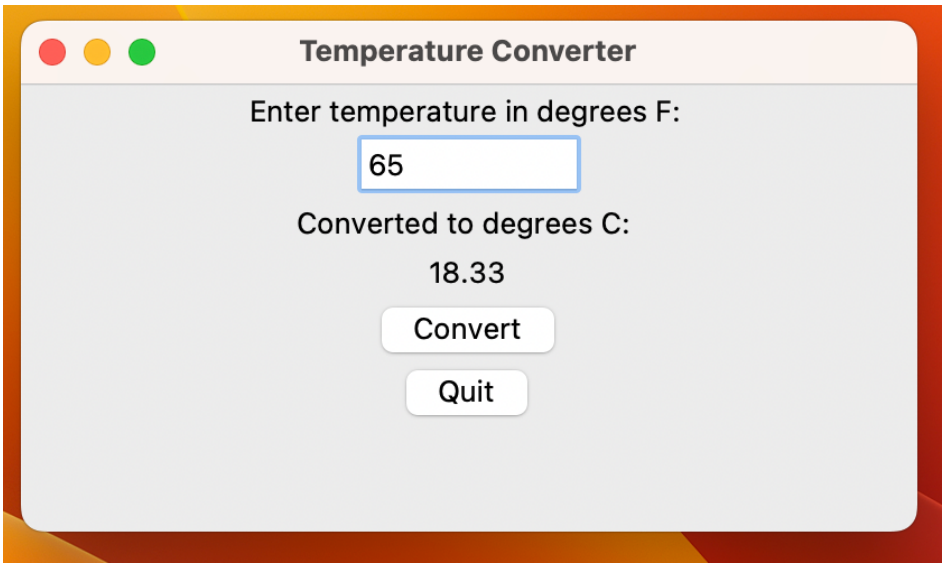
```
self.value = tk.StringVar()
self.deg_label = tk.Label(self.mid_frame, textvar = self.value)
```

Pack the objects and the new frame:

```
self.descrip_label.pack()
self.deg_label.pack()
self.mid_frame.pack()
```

In the convert method, store the calculated value in the StringVar object:

```
    def convert(self):
        fahr = float(self.temp_entry.get())
        cel = (fahr - 32) * 5/9
        cel = round(cel, 2)
        self.value.set(cel)
```



Other Features to Explore

- Radio buttons
- Check boxes
- Scrollable listboxes
- Canvas widget to draw shapes
- Menus
- Progress bars
- Top-level windows that keep the user from interacting with the main window
- Changing background and text color, font and font size
- tkinter.ttk themed widgets such as a combobox